BÀI TẬP TÌM KIẾM TRONG AI

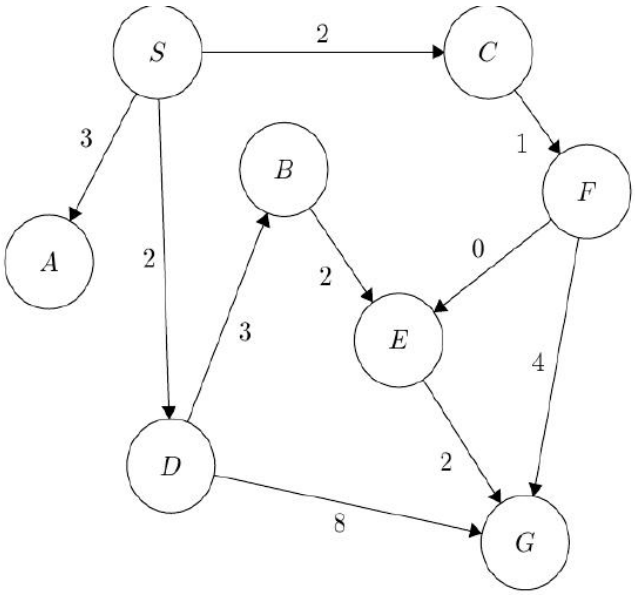
Họ và tên	**NGUYỄN LƯƠNG HỒNG BÚP**
MSSV / Lớp	**3124720009/ DNT1241**

Mục tiêu học tập

- Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
- Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
- Thiết kế và phân tích heuristic cho A*.
- So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
- Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

Các cạnh: S→A(3), S→C(2), S→D(2), C→F(1), D→B(3), D→G(8), B→E(2), F→E(0), F→G(4), E→G(2); A không có cạnh đi ra. Quy ước: graph search, kiểm tra đích khi lấy nút ra khỏi frontier.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả

Nội dung	Kết quả
Explored Set	**S, A, C, D, F, B, G**
Path	**S → D → G**
Total Cost	**8 + 2 = 10**

Diễn giải (frontier FIFO): lấy S → [A, C, D]; lấy A → [C, D]; lấy C → [D, F]; lấy D → [F, B, G]; lấy F → [B, G, E]; lấy B → [G, E]; lấy G = đích, dừng. Cost: S→D = 2, D→G = 8.

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	**S, A, C, F, E, G**
Path	**S → C → F → E → G**
Total Cost	**2 + 1 + 0 + 2 = 5**

Diễn giải (frontier LIFO, hòa chọn theo alphabet): S → A (ngõ cụt, quay lui) → C → F → E (E trước G) → G = đích, dừng.

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	**S**
1	**S, A, C, D**
2	**S, A, C, F, D, B, G (tìm thấy G, dừng)**
...	**Không cần chạy tiếp vì đã tìm thấy G ở giới hạn 2**

Path cuối cùng: S → D → G

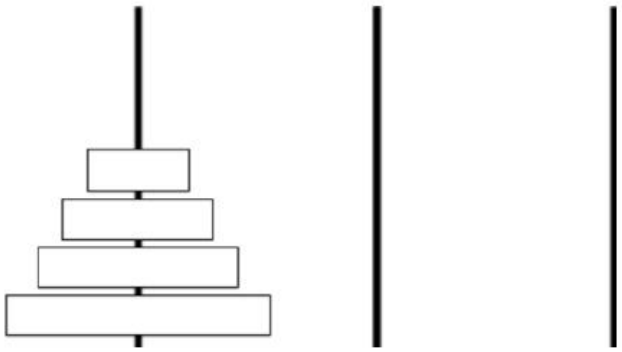
Total Cost: 2 + 8 = 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

Trả lời: BFS dùng frontier là hàng đợi FIFO nên duyệt theo từng tầng (nút gần S trước), tìm được đường có ít bước nhất; BFS chỉ tối ưu về chi phí khi mọi cạnh có chi phí bằng nhau (ở đây BFS cho cost 10, trong khi đường tối ưu có cost 5). DFS dùng frontier là ngăn xếp LIFO nên đi sâu hết một nhánh rồi mới quay lui; DFS không đảm bảo tối ưu (ở đây ra cost 5 chỉ do thứ tự duyệt thuận lợi) nhưng tốn ít bộ nhớ hơn BFS.

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Đánh số các đĩa từ 1 (nhỏ nhất) đến n (to nhất). Một trạng thái là bộ n số (p_1, p_2, ..., p_n), trong đó $p_i \in \{1, 2, 3\}$ là cọc mà đĩa i đang nằm. Ví dụ với 3 đĩa, (1, 3, 2) nghĩa là đĩa 1 ở cọc 1, đĩa 2 ở cọc 3, đĩa 3 ở cọc 2.

Biểu diễn này đủ vì trên mỗi cọc các đĩa luôn xếp theo thứ tự to dưới, nhỏ trên, nên chỉ cần biết đĩa nào ở cọc nào là suy ra được cấu hình của từng cọc và đĩa trên cùng của mỗi cọc.

2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Mỗi đĩa chọn độc lập 1 trong 3 cọc nên số trạng thái là $3 \times 3 \times ... \times 3$ (n lần) = 3^n. Mọi cách gán đều hợp lệ vì dù đĩa nào ở cọc nào cũng luôn xếp được to dưới, nhỏ trên. Ví dụ n = 3 có 27 trạng thái.

2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

S_0 = (1, 1, ..., 1): tất cả n đĩa đều nằm ở cọc 1.

2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Hành động là "chuyển đĩa trên cùng của cọc x sang cọc y", ký hiệu (x → y) với x ≠ y. Actions(s) gồm mọi (x → y) thỏa cả hai điều kiện: (1) cọc x không rỗng; (2) cọc y rỗng, hoặc đĩa trên cùng của cọc y to hơn đĩa trên cùng của cọc x (không đặt đĩa to lên đĩa nhỏ). Đĩa trên cùng của một cọc là đĩa có chỉ số i nhỏ nhất thỏa p_i = cọc đó.

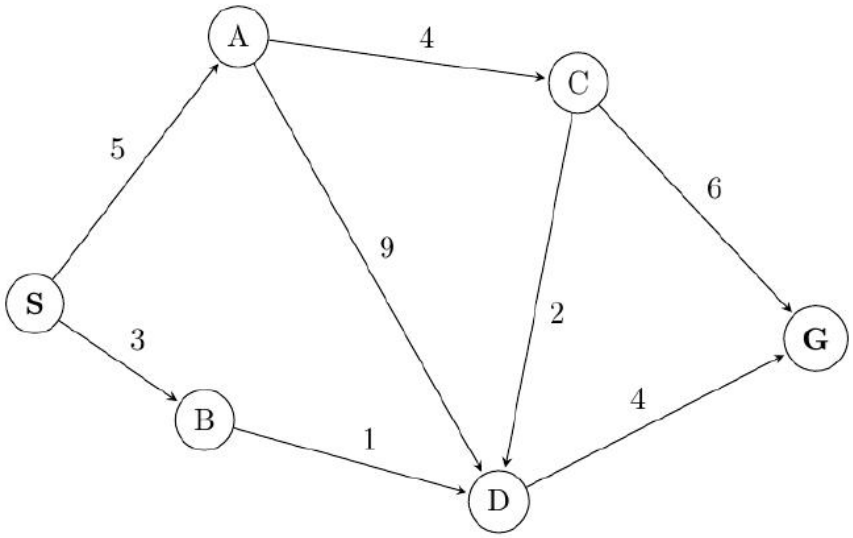
2.5. Goal Test

Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

goalTest(s) trả về True khi và chỉ khi mọi đĩa đều ở cọc thứ ba: $p_i = 3$ với mọi i, tức s = (3, 3, ..., 3). Ngược lại trả về False.

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

Các cạnh: S→A(5), S→B(3), A→C(4), A→D(9), B→D(1), C→D(2), C→G(6), D→G(4). Chi phí thật tới G: h(S)=8, h*(A)=10, h*(B)=5, h*(C)=6, h*(D)=4, h*(G)=0.*

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

Node	h(n)
S	**7**
A	**9**
B	**5**
C	**5**
D	**4**
G	**0**

Admissible: h(n) ≤ h(n) ở mọi nút. Consistent: h(n) ≤ c(n,n') + h(n') ở mọi cạnh: S→A: 7 ≤ 5+9; S→B: 7 ≤ 3+5; A→C: 9 ≤ 4+5; A→D: 9 ≤ 9+4; B→D: 5 ≤ 1+4; C→D: 5 ≤ 2+4; C→G: 5 ≤ 6+0; D→G: 4 ≤ 4+0.*

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S, B, D, G

Path: S → B → D → G

Total Cost: 3 + 1 + 4 = 8

Diễn giải: lấy S (f=7) → A (f=5+9=14), B (f=3+5=8); lấy B → D (g=4, f=4+4=8); lấy D → G (g=8, f=8); lấy G = đích, dừng.

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Chọn h(S) = 8, h(A) = 10, h(B) = 2, h(C) = 6, h(D) = 4, h(G) = 0. Heuristic này admissible vì h(n) ≤ h*(n) ở mọi nút (h*: S=8, A=10, B=5, C=6, D=4, G=0).

Cạnh vi phạm: S → B, vì h(S) = 8 > c(S,B) + h(B) = 3 + 2 = 5. Do đó h không consistent.

3.4. Câu hỏi mở rộng

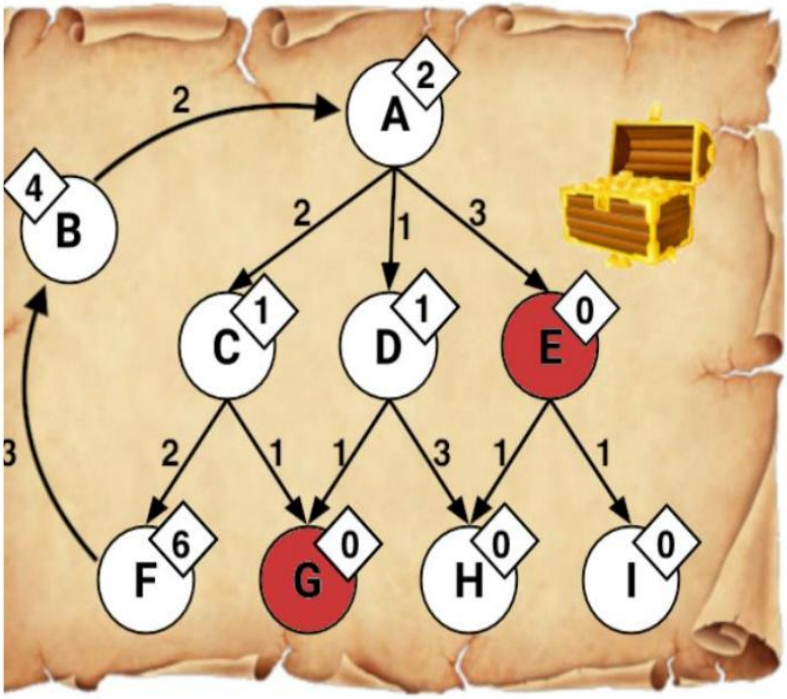
Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Giả sử h consistent, tức h(n) ≤ c(n,n') + h(n') với mọi cạnh (n,n'). Xét đường đi tối ưu từ n đến đích: n = n_0 → n_1 → ... → n_k = G. Áp dụng bất đẳng thức cho từng cạnh: h(n_0) ≤ c(n_0,n_1) + h(n_1) ≤ c(n_0,n_1) + c(n_1,n_2) + h(n_2) ≤ ... ≤ c(n_0,n_1) + ... + c(n_{k-1},n_k) + h(G).

Vì h(G) = 0 nên h(n) ≤ tổng chi phí của đường đi tối ưu = h*(n). Vậy h admissible. (Chiều ngược lại không đúng: admissible chưa chắc consistent, xem 3.3.)

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

Các cạnh: A→C(2), A→D(1), A→E(3), B→A(2), C→F(2), C→G(1), D→G(1), D→H(3), E→H(1), E→I(1), F→B(3). Kho báu: E và G. Heuristic: A=2, B=4, C=1, D=1, E=0, F=6, G=0, H=0, I=0.

4.1. Depth-First Search

Explored Set: A, C, F, B, G

Path returned: A → C → G (cost 3)

Diễn giải: A → C (C trước D, E) → F (F trước G) → B (con của B là A đã explored) → G = kho báu, dừng.

4.2. Breadth-First Search

Explored Set: A, C, D, E

Path returned: A → E (cost 3)

Diễn giải: lấy A → [C, D, E]; lấy C → [D, E, F, G]; lấy D → [E, F, G, H]; lấy E = kho báu, dừng.

4.3. Uniform-Cost Search

Explored Set: A, D, C, G

Path returned: A → D → G

Cost: 1 + 1 = 2

Diễn giải: lấy A (g=0) → C(2), D(1), E(3); lấy D → G(2), H(4); C và G cùng g=2, chọn C theo alphabet → F(4); lấy G (g=2) = kho báu, dừng.

4.4. Greedy Search

Explored Set: A, E

Path returned: A → E

Cost: 3

Diễn giải: lấy A → C(h=1), D(h=1), E(h=0); chọn E (h nhỏ nhất) = kho báu, dừng.

4.5. A* Search

Explored Set: A, D, G

Path returned: A → D → G

Cost: 2

Diễn giải: lấy A (f=2) → C(f=3), D(f=2), E(f=3); lấy D → G(f=2), H(f=4); lấy G (f=2) = kho báu, dừng.

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu (sâu nhất trước)	**A → C → G**	**3**
BFS	Độ sâu nhỏ nhất	**A → E**	**3**
UCS	g(n)	**A → D → G**	**2**
Greedy	h(n)	**A → E**	**3**
A*	g(n)+h(n)	**A → D → G**	**2**

Nhận xét: UCS và A cho đường tối ưu (cost 2); DFS, BFS và Greedy cho cost 3. A* mở rộng ít nút hơn UCS (3 nút so với 4 nút) nhờ heuristic.*

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

Đúng / Sai: Sai

Giải thích: DFS có thể may mắn đi thẳng tới đích và mở rộng ít nút hơn A*. Phản ví dụ: S có hai con là A (cost 1, A là ngõ cụt) và G (cost 10, là đích). Nếu DFS chọn G trước thì chỉ mở rộng S rồi lấy G ra làm đích; còn A* với h = 0 phải mở rộng S rồi A (f = 1) trước khi lấy G (f = 10). Vậy DFS mở rộng ít nút hơn A*.

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

Đúng / Sai: Sai

Giải thích: Xe đi được nhiều ô trong một nước nên Manhattan distance có thể đánh giá quá cao số nước đi. Phản ví dụ: hai ô cùng hàng cách nhau 7 ô (bàn trống) có Manhattan distance = 7 nhưng chỉ cần 1 nước đi (7 > 1), nên không admissible.

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

Đúng / Sai: Đúng

Giải thích: Pacman di chuyển trên lưới theo các bước nằm ngang/dọc, nên mọi đường đi thực tế có độ dài không nhỏ hơn đường thẳng nối hai điểm. Khoảng cách Euclid không bao giờ vượt quá chi phí thật nên admissible.

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Tổng có thể vượt chi phí thật. Phản ví dụ: nếu $h_1 = h_2 = h^* > 0$ (cả hai đều admissible) thì $h_1 + h_2 = 2h^* > h^*$, nên tổng không admissible.

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích: Admissible không kéo theo consistent (chiều ngược lại mới đúng: consistent ⇒ admissible). Phản ví dụ: heuristic ở câu 3.3 là admissible nhưng không consistent vì h(S) = 8 > c(S,B) + h(B) = 3 + 2 = 5.

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: Với graph search, A* chỉ đảm bảo tối ưu khi heuristic consistent (admissible là đủ cho tree search). Phản ví dụ: S→A(1), S→B(1), A→C(1), B→C(2),

C→G(3); h(A) = 4, các nút còn lại h = 0. Heuristic admissible nhưng không consistent (h(A) = 4 > 1 + h(C) = 1). A* lấy B (f=1), rồi C (g=3, f=3) và đóng C; sau đó gặp C qua A (g=2) nhưng bỏ qua vì C đã explored, nên trả về S→B→C→G với cost 6, trong khi đường tối ưu S→A→C→G có cost 5.

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).

---o0o---
(Hết)